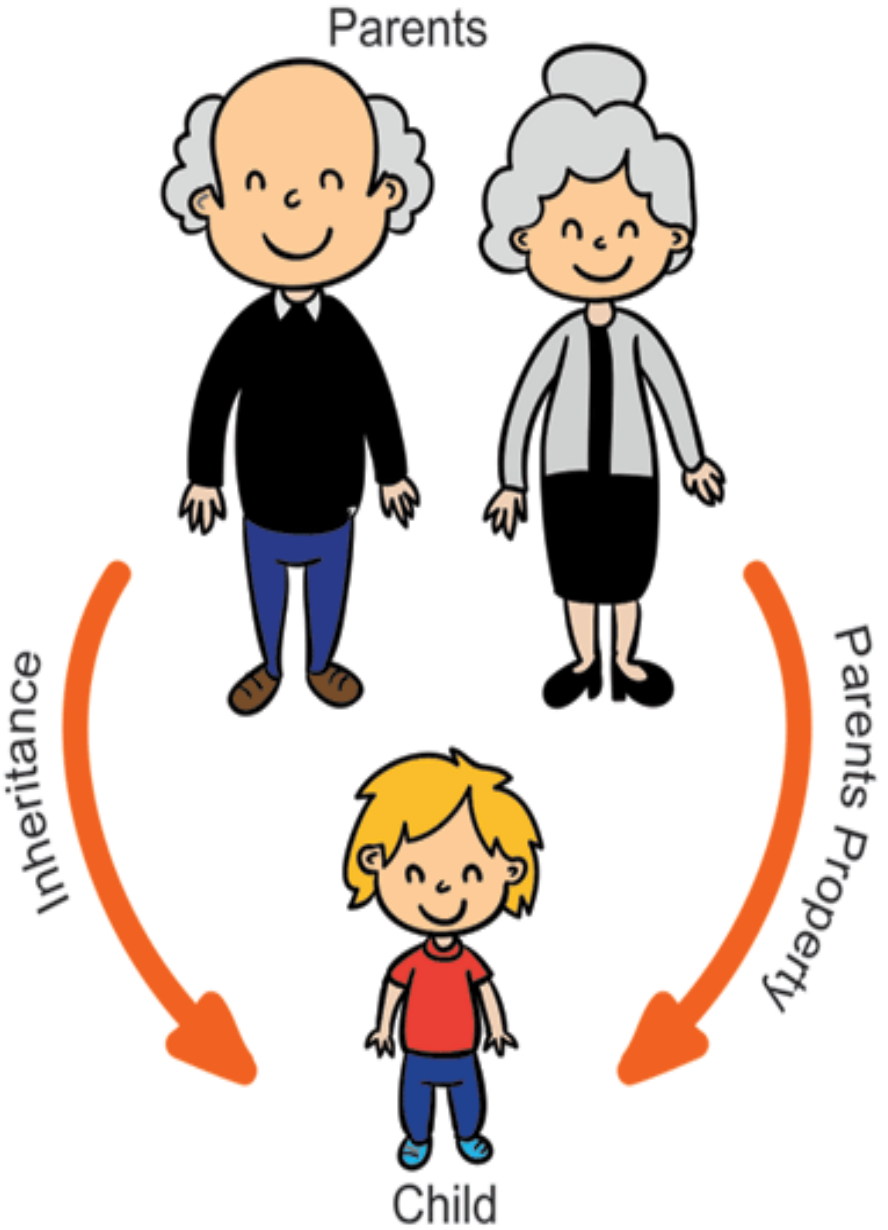

Class Dog
Extends
Animal
Class
Animal
INHERITANCE



Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object.

Reuse of code. Instead of using the same set of code multiple times we can create another class from an existing class.

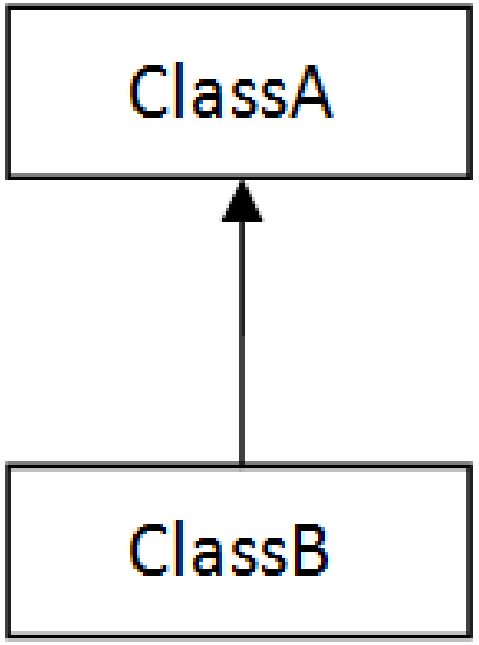
- We can create new classes that are built upon existing classes.
- When we inherit from an existing class, we can reuse methods and fields of the parent class.
- We can add new methods and fields in your current class also.

Terms used in Inheritance

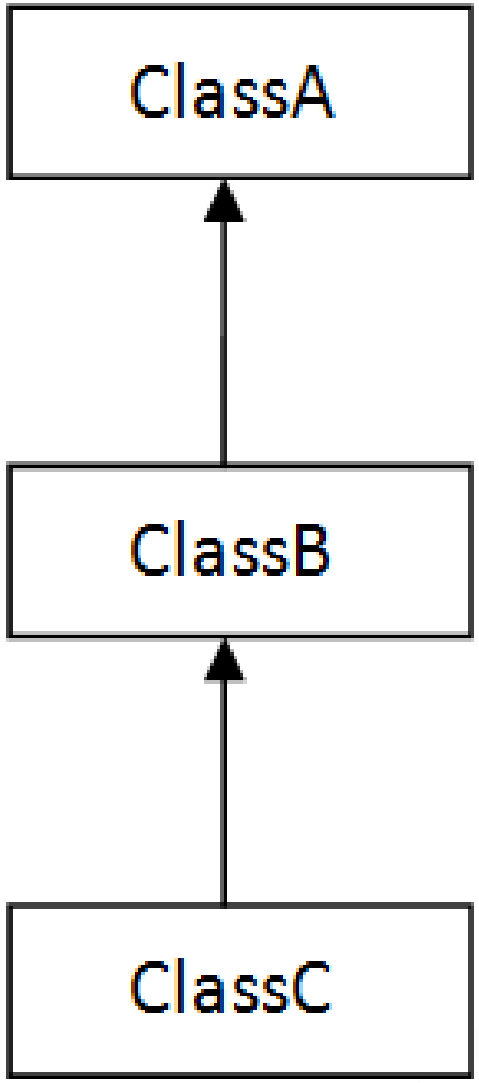
- **Class:** A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.
- **Sub Class/Child Class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- **Super Class/Parent Class:** Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.
- **Reusability:** As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defined in the previous class.

The syntax of Java Inheritance

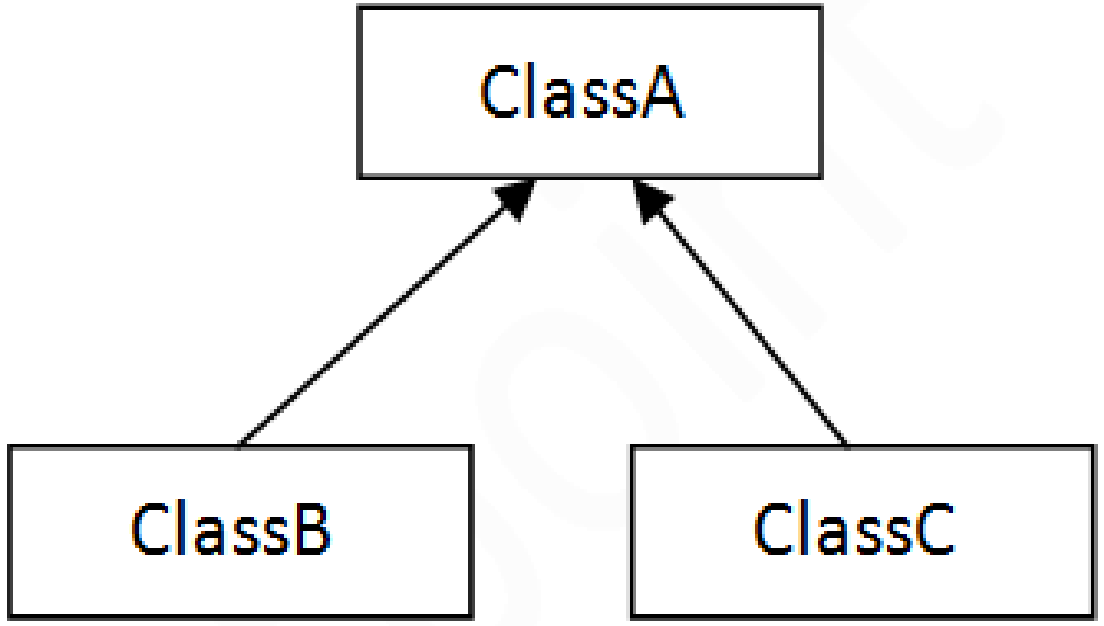
```
1.class Subclass-name extends Superclass-name

2.{

3.   //methods and fields

4.}
```



1) Single



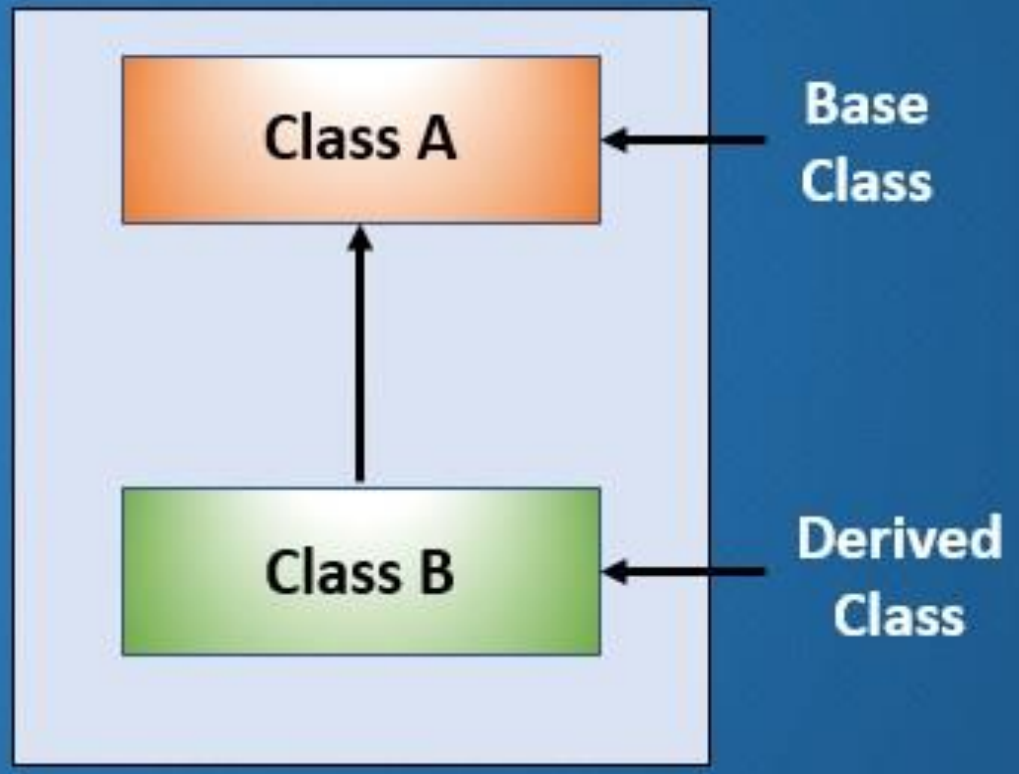
2) Multilevel



3) Hierarchical

A subclass inherits from one superclass.

Single Inheritance in Java



```
class Animal
{
  // methods and fields
}
// use of extends keyword
// to perform inheritance

class Dog extends Animal
 {
  // methods and fields of Animal
  // methods and fields of Dog
}
```

```
class Animal{
void eat(){
System.out.println("eating...");  }
}
class Dog extends Animal{
void bark(){
System.out.println("barking...");}  }
class TestInheritance{
public static void main(String args[]){
Dog d=new Dog();
d.bark();
d.eat();
}}
```

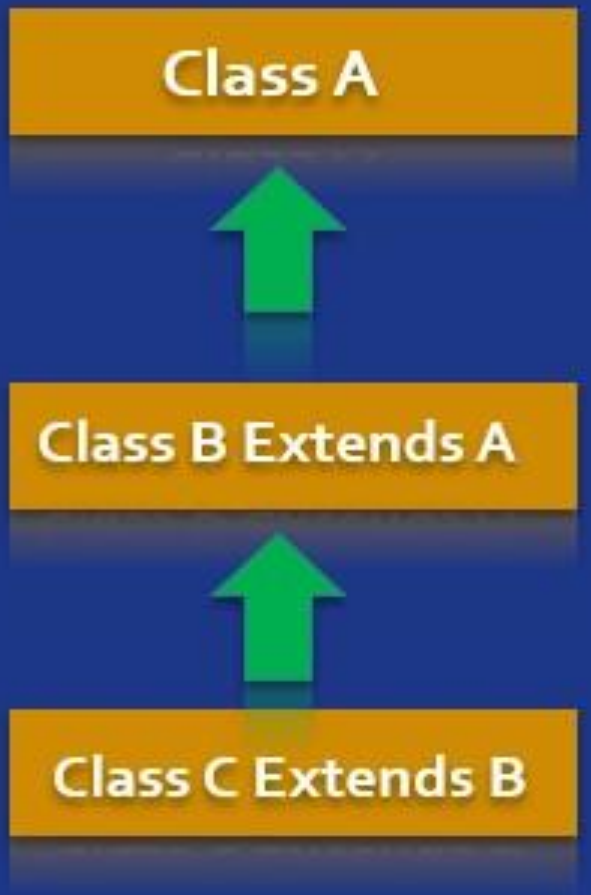
```
class Parent {
    int a = 20;
    void display() {
        System.out.println("PARENT");
    }
}

class Child extends Parent {
    int b = 10;
    void show() {
        System.out.println("CHILD");
    }
}
```

```
public class Inherit {
    public static void main(String[] args)
    {
        Child c = new Child();
        System.out.println(c.b);
        c.show();
        System.out.println(c.a);
        c.display();
    }
}
```

```
10
CHILD
20
PARENT
```

Multilevel Inheritance in Java

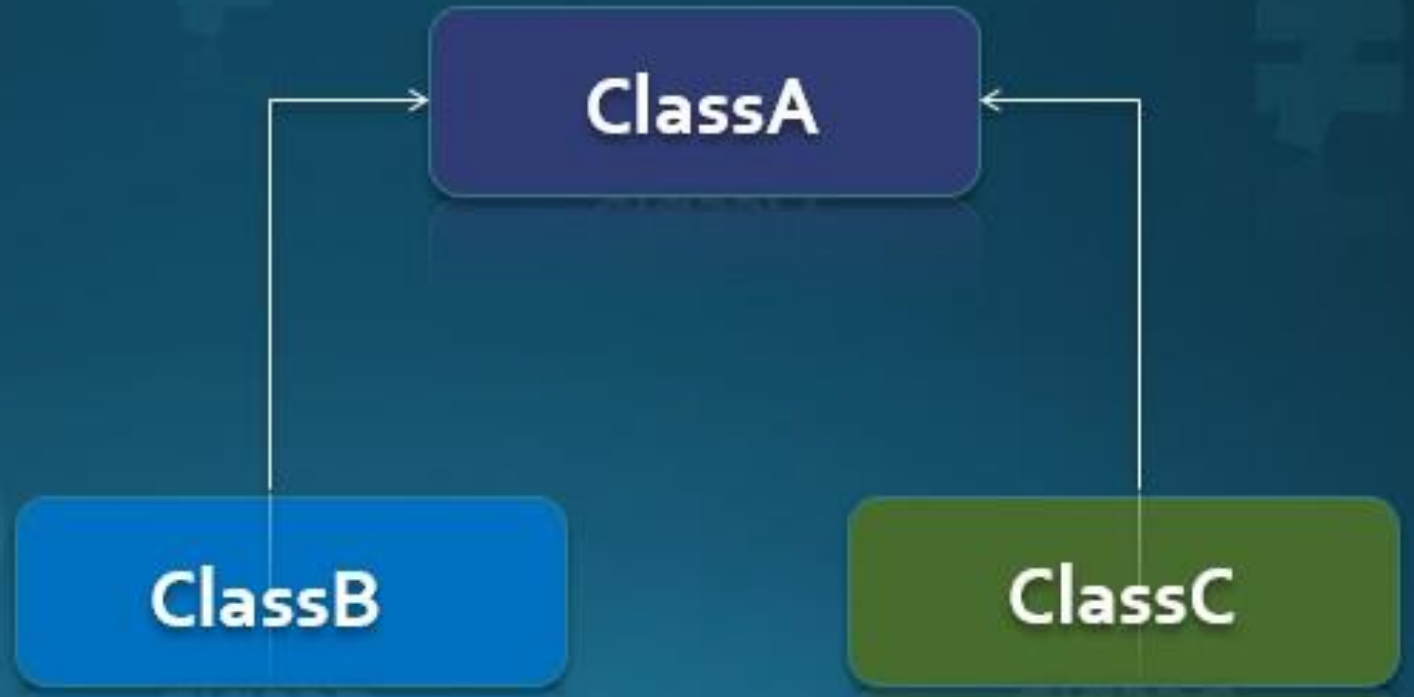
A class inherits from a subclass, making it a subclass of a subclass.



```
class Animal {
   void eat() {
      System.out.println("Eating...");
   }
}
class Dog extends Animal {
   void bark() {
      System.out.println("Barking...");
   }
}
class BabyDog extends Dog { // Inherits from Dog class
   void weep() {
      System.out.println("Weeping...");
   }
}
public class TestInheritance {
   public static void main(String args[]) {
      BabyDog bd = new BabyDog();
      bd.weep();
      bd.bark();
      bd.eat();
   } }
```

Hierarchical Inheritance in Java

Multiple subclasses inherit from one superclass.



Java

```
class Animal {
   void eat() {
      System.out.println("Animal is eating");
   }
}
class Dog extends Animal {
   void bark() {
      System.out.println("Dog is barking");
   }
}
class Cat extends Animal { // Inherits from Animal
   void meow() {
      System.out.println("Cat is meowing");
   }
}
public class TestInheritance {
   public static void main(String args[]) {
      Dog d = new Dog();
      Cat c = new Cat();
      d.eat();
      d.bark();
      c.eat();
      c.meow();
   } }
```

Super keyword in java

Super Keyword in Java

- The **super** keyword in Java is a reference variable which is used to refer immediate parent class object.
- Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by super reference variable.

Usage of Java super Keyword

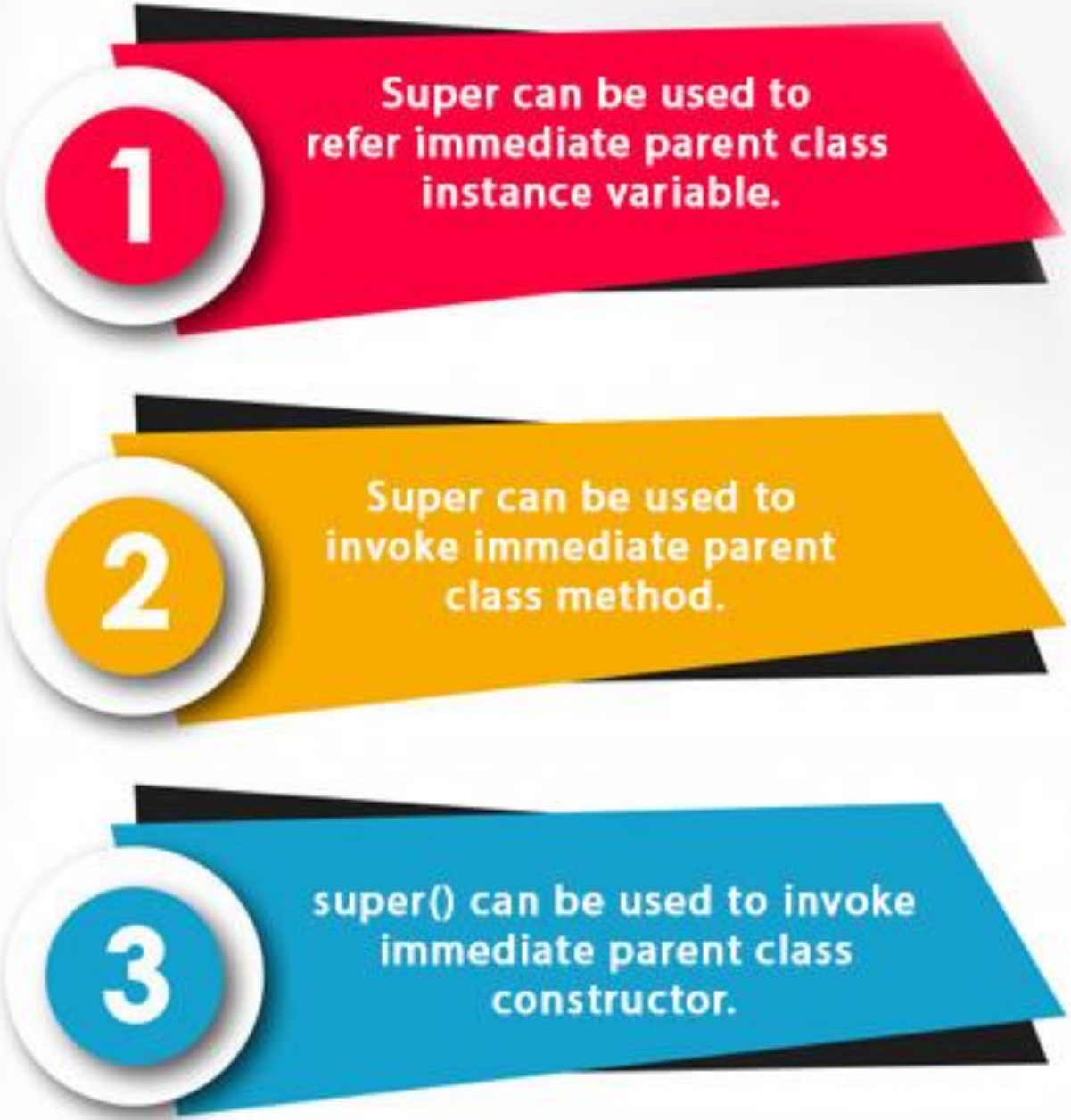
1.super can be used to refer immediate parent class instance variable.

2.super can be used to invoke immediate parent class method.

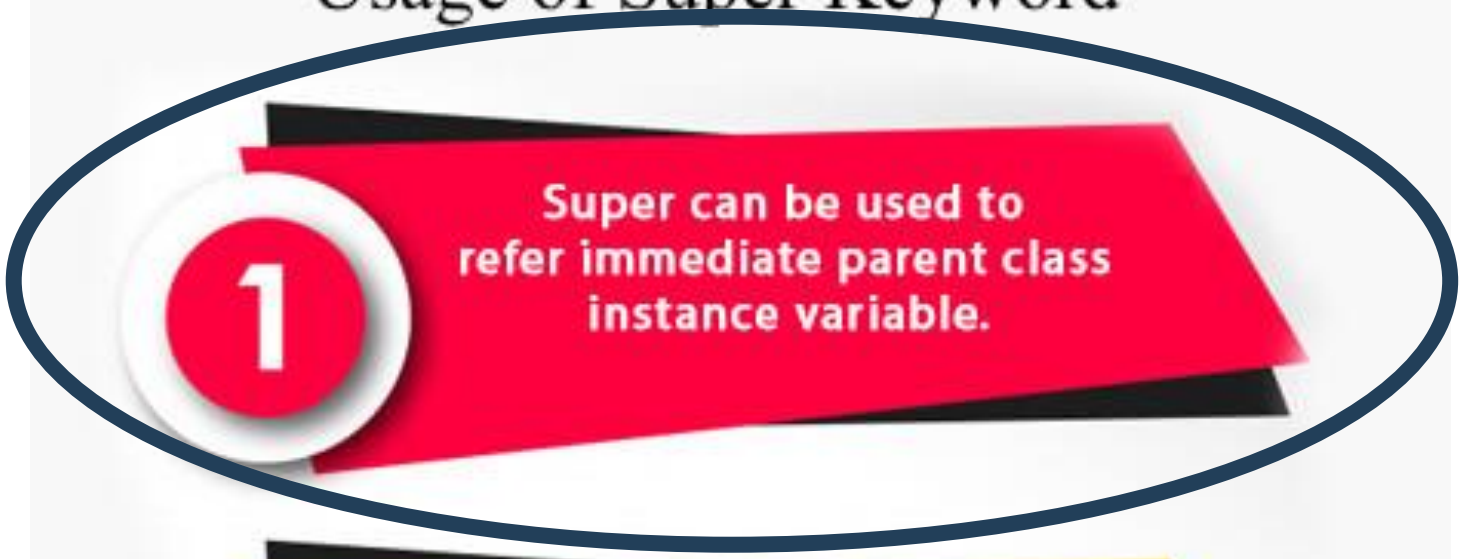
3.super() can be used to invoke immediate parent class constructor.

```
class Animal{
String color="white";
}
class Dog extends Animal{
String color="black";
void printColor(){
System.out.println(color);//prints color of Dog class
System.out.println(super.color);//prints color of Animal class
}
}
class TestSuper1{
public static void main(String args[]){
Dog d=new Dog();
d.printColor();
}}
```

Usage of Super Keyword

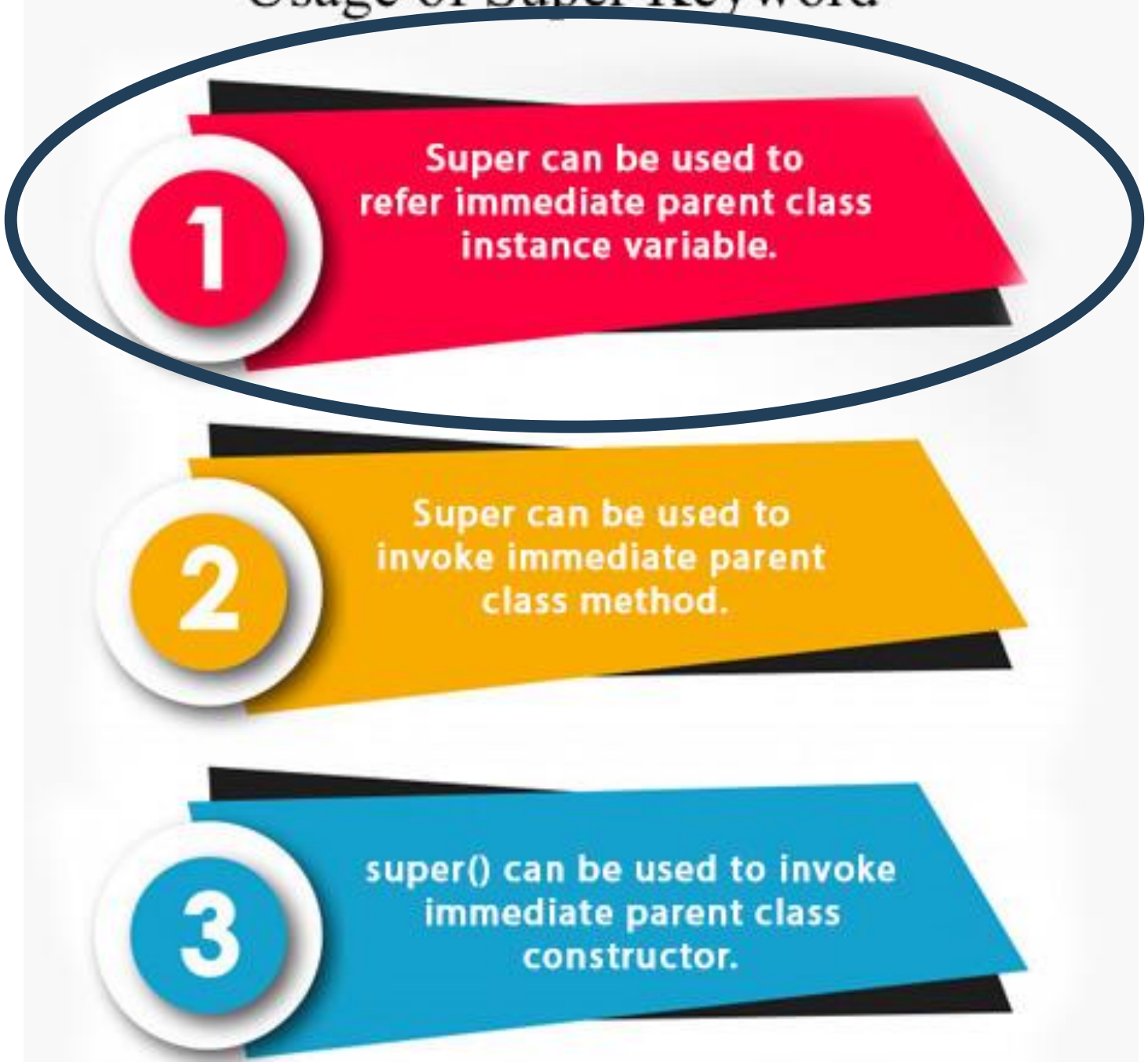


Usage of Super Keyword

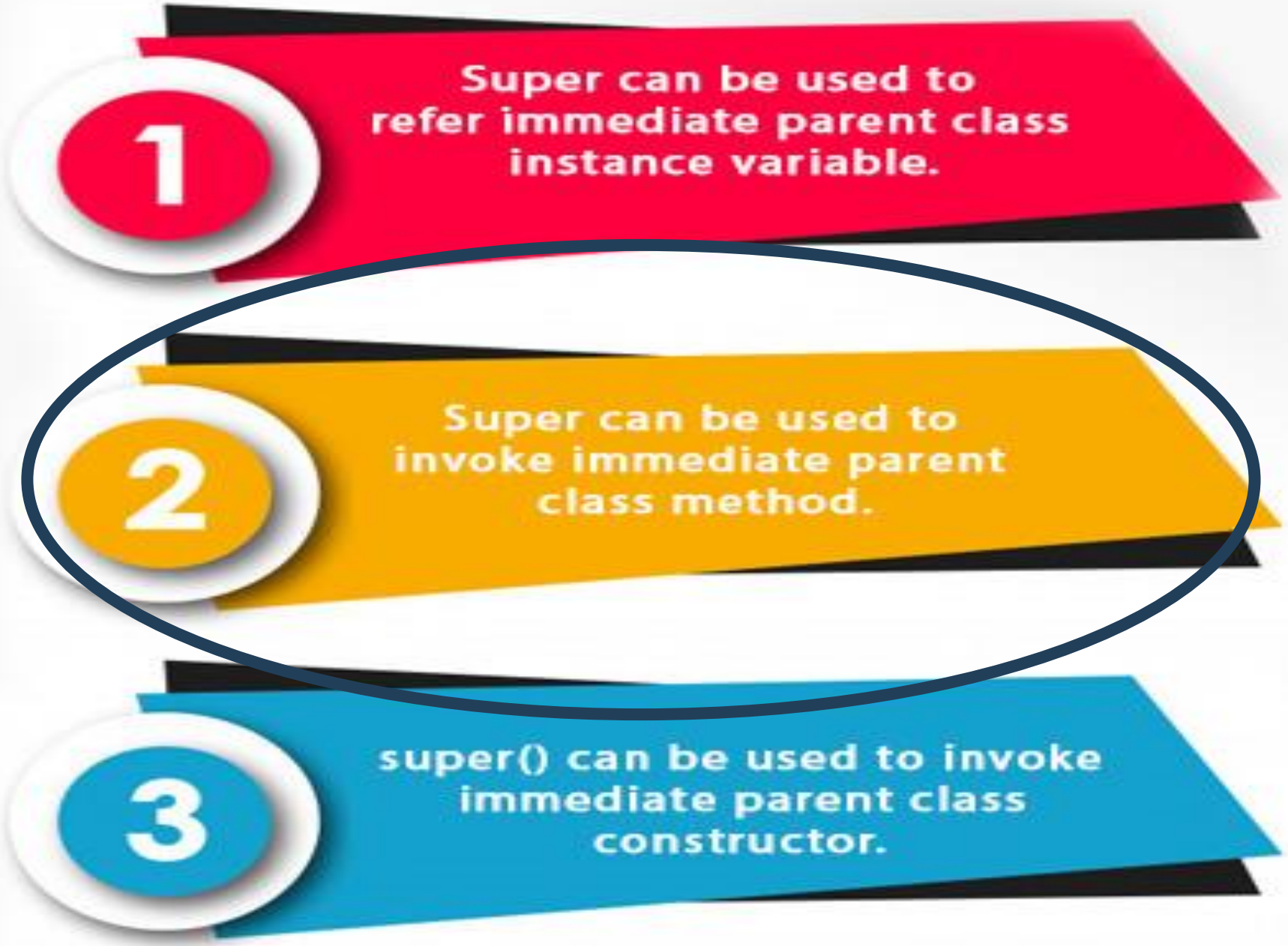


```
class Vehicle {
   int speed = 50;
}
class Car extends Vehicle {
   int speed = 100;

   void display() {
      System.out.println("Speed of the Vehicle is: " + super.speed + " km/h");
      System.out.println("Speed of the Car is: " + speed + " km/h");
   }
}
class TestSuperVariable1 {
   public static void main(String args[]) {
      Car smallCar = new Car();
      smallCar.display();
   }
}
```

Usage of Super Keyword

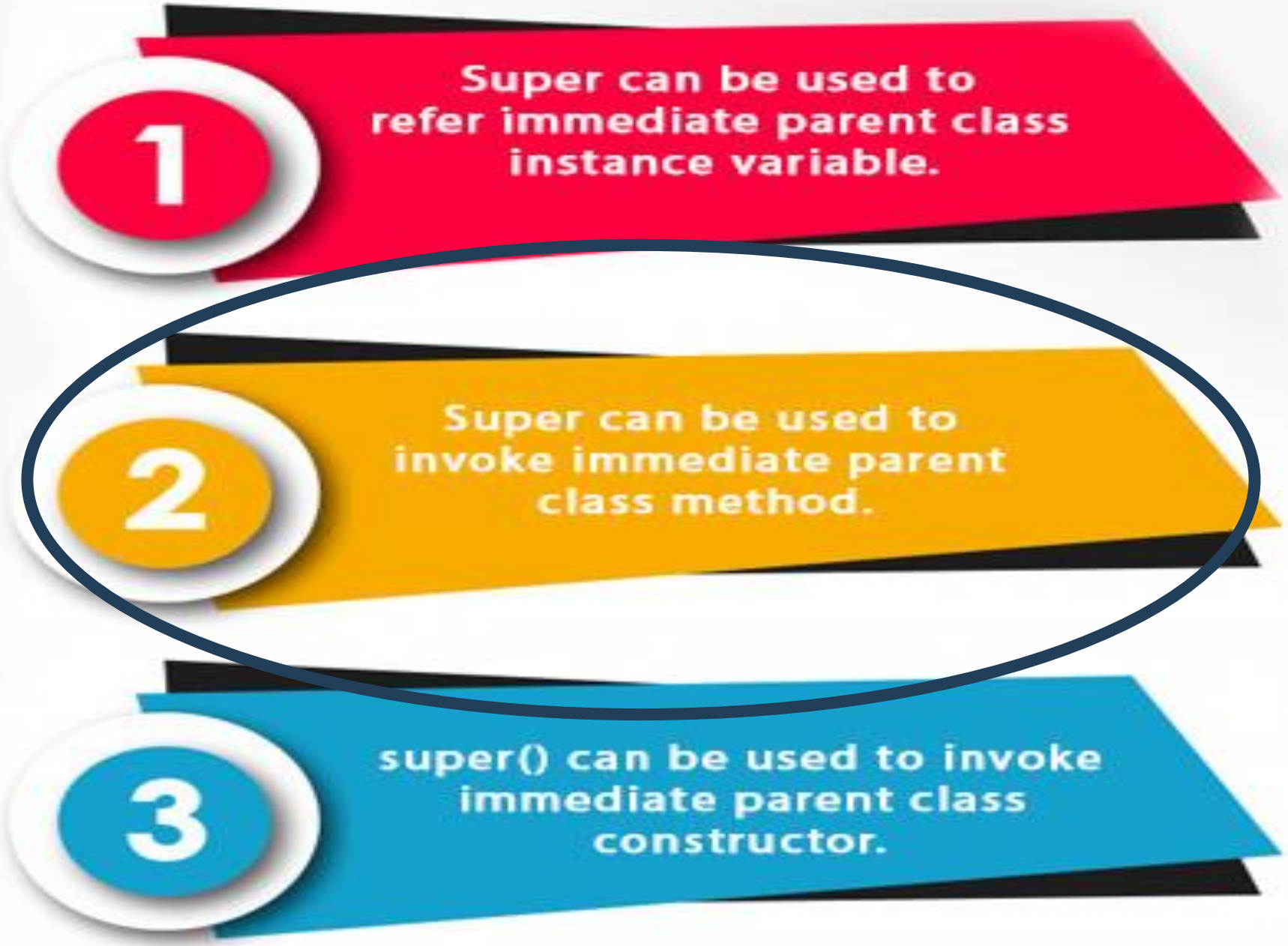


Usage of Super Keyword

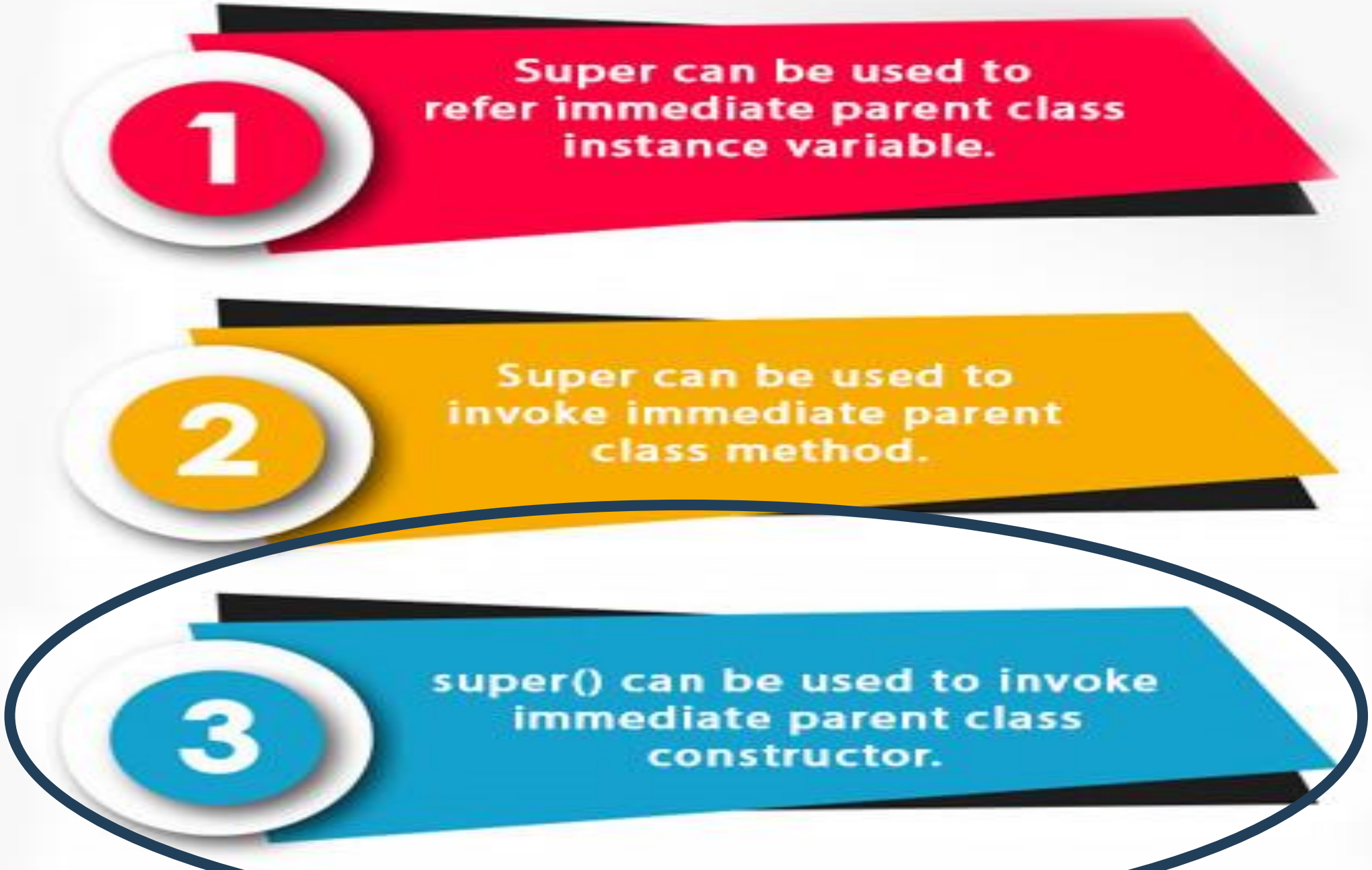


```
class Parent {
    void show() {
        System.out.println("This is the parent class show method.");
    }
}

class Child extends Parent {
    void show() {
        super.show(); // Invokes the show method of the parent class
        System.out.println("This is the child class show method.");
    }
}

public class TestSuperMethod1 {
    public static void main(String[] args) {
        Child obj = new Child();
        obj.show();
    }
}
```

Usage of Super Keyword



Usage of Super Keyword



```
class Animal {
    Animal() {
        System.out.println("Animal is created");
    }
}

class Dog extends Animal {
    Dog() {
        super(); // Invoking the parent class constructor
        System.out.println("Dog is created");
    }
}

public class TestSuperConstructor1 {
    public static void main(String args[]) {
        Dog d = new Dog();
    }
}
```

Final Keywords in Java

TechVidvan

The final keyword in java is used to restrict the user. The java final keyword can be used in many context.

Final can be:

1.variable

2.method

3.Class

The final keyword can be applied with the variables, a final variable that have no value it is called blank final variable or uninitialized final variable. It can be initialized in the constructor only. The blank final variable can be static also which will be initialized in the static block only.

1) Java final variable

If you make any variable as final, you cannot change the value of final variable(It will be constant).

Example of final variable

There is a final variable named speedLimit. We are attempting to change the value of this variable, but it cannot be changed because once a final variable is assigned a value, it can never be modified.

```
class Bike9{
 final int speedlimit=90;//final variable
 void run(){
  speedlimit=400;
 }
 public static void main(String args[]){
 Bike9 obj=new  Bike9();
 obj.run();
 }
}//end of class
```

Output:Compile Time Error

2) Java final method

If you make any method as final, you cannot override it.

Example of final method

Output:Compile Time Error

```
class Bike{
  final void run(){System.out.println("running");}
}

class Honda extends Bike{
   void run(){System.out.println("running safely with 100kmph");}

   public static void main(String args[]){
   Honda honda= new Honda();
   honda.run();
   }  }
```

3) Java final class

If you make any class as final, you cannot extend it.

Example of final class

Output:Compile Time Error

```
final class Bike{}

class Honda1 extends Bike{
  void run(){System.out.println("running safely with
100kmph");}

  public static void main(String args[]){
  Honda1 honda= new Honda1();
  honda.run();
  }
}
```

Is final method inherited?

Ans) Yes, final method is inherited but you cannot override it. For Example:

Output:running...

```
class Bike{
  final void run(){System.out.println("running...");}
}
class Honda2 extends Bike{
   public static void main(String args[]){
    new Honda2().run();
   }
}
```

- **What is blank or uninitialized final variable?**

- **A final variable that is not initialized at the time of declaration is known as blank final variable.**

- **If you want to create a variable that is initialized at the time of creating object and once initialized may not be changed, it is useful. For example PAN CARD number of an employee.**

- **It can be initialized only in constructor.**

- **Example of blank final variable**
- **class Student{**
- **int id;**
- **String name;**
- **final String PAN_CARD_NUMBER;**
- **...**
- **}**

Can we initialize blank final variable?

Yes, but only in constructor. For example:

```
class Bike10{
  final int speedlimit;//blank final variable

  Bike10(){
  speedlimit=70;
  System.out.println(speedlimit);
  }

  public static void main(String args[]){
    new Bike10();
 }
}
```

Output: 70

Static blank final variable

A static final variable that is not initialized at the time of declaration is known as static blank final variable. It can be initialized only in static block.

Example of static blank final variable

```
class A{
  static final int data;//static blank final variable
  static{ data=50;}
  public static void main(String args[]){
    System.out.println(A.data);
 }
}
```

What is final parameter?

If you declare any parameter as final, you cannot change the value of it.

Output: Compile Time Error

```
class Bike11{
  int cube(final int n){
   n=n+2;//can't be changed as n is final
   n*n*n;
  }
  public static void main(String args[]){
    Bike11 b=new Bike11();
    b.cube(5);
 }
}
```

Can we declare a constructor final?

- No, because constructor is never inherited.